

算法驱动的迷宫探险游戏开发方法

张佳栋 方维 逯熙钰 姜舜航

(东北大学计算机科学与工程学院 沈阳 110169)

摘 要 本文提出了一种融合经典算法策略的迷宫探险游戏开发方法, 涵盖从迷宫生成到智能策略执行的完整流程。首先, 基于分治法生成满足“无孤立区域存在唯一通路”的随机迷宫, 支持多种尺寸, 并实现了游戏过程的 OpenGL 动态可视化。其次, 采用树形动态规划实现全局资源收集路径规划, 确保在避开陷阱的前提下最大化资源总值; 实验中, 该方法在 100 组 7×7 迷宫中的平均资源得分为 6.72。针对有限视野场景, 设计了一种贪心策略和启发式策略, 通过预期收益函数实现局部最优选择, 平均得分达 6.18, 较普通贪心策略 (4.89) 提高 26.3%。此外, 利用回溯法构建密码解谜系统, 在限制条件下成功解出所有 3 位数谜题; 在 BOSS 战斗模块中, 基于分支限界法实现技能序列优化策略, 在可用资源有限的情况下稳定输出最小回合击败序列。整体框架展示了各类算法在路径规划、决策优化和策略推理中的有效结合, 为算法驱动的游戏开发提供了实用范式。

关键词 迷宫生成; 树形 DP; 贪心策略; 启发式策略; 回溯法; 分支限界法; 可视化
中图法分类号 TP

Algorithm-Driven Maze Exploration Game Development Method

ZHANG Jia-Dong FANG Wei LU Xi-Yu JIANG Shun-Hang

(School of Computer Science and Engineering, Northeastern University, Shenyang 110169, China)

Abstract This paper presents a maze exploration game development framework integrating classical algorithmic strategies, covering the entire process from maze generation to intelligent strategy execution. First, a divide-and-conquer approach is used to generate random mazes that guarantee unique solvable paths without isolated regions. The system supports multiple maze sizes and features real-time OpenGL-based visualization. Second, a tree-based dynamic programming method is adopted for global resource collection path planning, ensuring maximum total value while avoiding traps. In experiments on 100 randomly generated 7×7 mazes, this method achieved an average resource score of 6.72%. For limited-vision scenarios, both a greedy strategy and a heuristic strategy are designed. A reward expectation function guides local optimal selection, resulting in an average score of 6.18 with an improvement of 26.3% over the basic greedy approach (4.89). Additionally, a backtracking algorithm is employed to construct a password puzzle system, successfully solving all 3-digit puzzles under given constraints. In the boss battle module, a branch-and-bound method is used to optimize the skill sequence, consistently generating minimal-turn victory strategies under limited player resources. This framework demonstrates an effective integration of various algorithms in path planning, decision-making, and strategic reasoning, offering a practical paradigm for algorithm-driven game development.

Keywords maze generation; tree-based DP; greedy strategy; heuristic strategy; backtracking; branch and bound; visualization

1 项目概述

本项目旨在开发一款融合经典算法与图形可视化的迷宫探险游戏. 通过将分治、动态规划、贪心、回溯、分支限界等算法设计策略贯穿于迷宫生成、资源收集、谜题解锁、战斗系统等多个关键模块, 实现算法理论与工程实践的深度融合.

项目主要有以下模块:

- (1) 分治法生成随机迷宫
- (2) 动态规划求解最优资源收集路径
- (3) 贪心算法实时策略模拟
- (4) 启发式算法实时策略模拟
- (5) 回溯法解锁机关密码
- (6) 分支限界法优化 BOSS 战斗策略
- (7) 基于 OpenGL 的可视化演示

2 分治法地图生成

2.1 问题描述

任务: 使用分治法生成迷宫, 无孤立区域, 存在唯一通路.

输入: 迷宫尺寸.

输出: 迷宫矩阵 (可存储为 JSON 或 CSV), 包括起点 Start(S)、终点 Exit(E)、墙壁 (#)、通路 (空格)、资源 (例如金币 G)、陷阱 Trap(T)、机关 Locker(L)、BOSS(B).

2.2 算法设计

1. 初始化迷宫

创建二维数组表示迷宫, 尺寸为奇数 (如 $height \times width$).

2. 递归分割区域

输入参数: 子问题迷宫的左上方坐标和右下方坐标.

终止条件: 区域宽度或高度小于 2.

3. 选择分割方向

随机选择一个纵向墙体(非外围) 和一个横向墙体(非外围), 视其把迷宫分割为四个部分作为其子问题并求解, 同时在四片墙体中随机选取三个墙体将其打通, 由于 2×2 迷宫的特殊性, 这样生成的迷宫路径唯一且无环路, 且无需合并子问题.

4. 设置入口和出口

入口: 在迷宫最外围墙体(非四个角) 上随机选择一个作为入口.

出口: 在迷宫最外围墙体(非四个角) 上随机选择一个作为出口, 若与出口重合, 则重新生成直到不同.

5. 添加资源和陷阱

随机在迷宫中添加资源和陷阱, 数量和位置可根据需求设定.

2.3 复杂度分析

下面用归纳法证明对于生成一个 $a \times b$ 的迷宫的时间复杂度是 $\mathcal{O}(a \times b)$.

由于最小子问题是一个 $1 \times n$ 或 $n \times 1$ 大小的块, 处理这样一个子问题的复杂度是 $\mathcal{O}(n)$ 的, 满足归纳假设.

假设对于所有 $a < A, b < B$ 的问题的复杂度 $T(a, b) = \mathcal{O}(a \times b)$.

对于一个 $A \times B$ 的问题, 假设划分大小为 p, q . 划分的复杂度为 $\mathcal{O}(1)$.

那么复杂度

$$\begin{aligned} T(A, B) &= \mathcal{O}(1) + T(p, q) + T(p, B - q) + \\ &\quad + T(A - p, q) + T(A - p, B - q) \\ &= \mathcal{O}(1) + \mathcal{O}(p \times q) + \mathcal{O}(p \times (B - q)) + \\ &\quad + \mathcal{O}((A - p) \times q) + \mathcal{O}((A - p) \times (B - q)) \\ &= \mathcal{O}(A \times B). \end{aligned}$$

所以, 规模为 $a \times b$ 的问题的时间复杂度是 $\mathcal{O}(a \times b)$.

特别的, 当 $a = b = n$ 时, 复杂度为 $\mathcal{O}(n^2)$.

2.4 伪代码

见附录 1.

3 动态规划求解最优路径

3.1 问题描述

任务: 计算从起点到终点的最优资源收集路径, 避开陷阱, 优先拾取资源. 例如: 状态 $dp[i][j]$ 表示走到坐标 (i, j) 时的最大资源值.

输入: 迷宫矩阵、资源分布 (例如金币 = 5)、陷阱位置 (可设置陷阱 = -3).

输出: 最大资源值、最优路径序列.

需求: 路径可视化 (可选).

3.2 问题分析

目标: 从迷宫的起点出发, 到达终点, 经过必须经过的机关和 BOSS, 收集最多的资源.

分析: 地图保证了路径唯一, 可知地图实际是一个树形结构, 可以采用树形 DP. 我们以起点为根, 构建地图对应的树. 问题就转化为了有一颗带权树, 要求找一条从根到终点的通路, 使得通路上的权值和最大.

3.3 状态设计及状态转移方程

定义 $dp_{i,j}$ 表示从点 (i, j) 出发能获得的最大价值, $must_{i,j}$ 表示点 (i, j) 是否为从起点到终点必须经过的点. 那么答案即为 dp_s , 其中 s 为起点的坐标. 那么有转移方程:

$$\text{must}_p = \begin{cases} \bigvee_{x \in \text{son}_p} \text{must}_x, & \text{if } x \text{ is not E} \\ 1, & \text{if } x \text{ is E} \end{cases}$$

$$\text{dp}_p = w_p + \sum_{x \in \text{son}_p} \begin{cases} \text{dp}_x, & \text{if } \text{must}_x \\ \max(\text{dp}_x, 0), & \text{else} \end{cases}$$

3.4 最优路径构建

对于不同 must 的点的构建方法不同.

对于 must 为真的节点, 需要先输出自己的坐标, 然后依次构建 must 为假并且采用其中权值的子树的路径, 中间穿插自己的坐标, 最后构建 must 为真的子树的路径.

对于 must 为假的节点, 也需要先输出自己的坐标, 然后依次构建采用其中权值的子树的路径, 中间穿插自己的坐标, 并且最后也要输出自己的坐标.

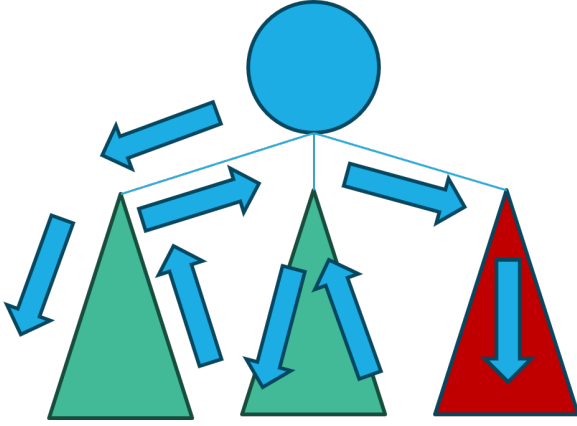


图1 must 为真的节点的子树路径构建顺序

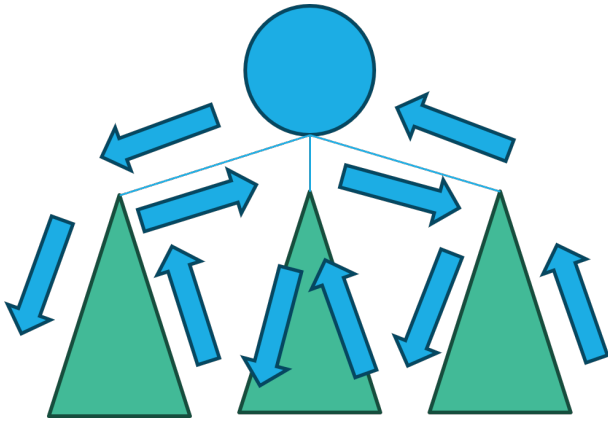


图2 must 为假的节点的子树的路径构建顺序

3.5 复杂度分析

本算法采用自底向上的树形动态规划方法, 其中每个点 p 只会在其父节点的转移中被访问一次, 而 must 的计算也是在一次 DFS 遍历中完成的.

首先, 自底向上计算每个点是否为必须经过的点, 即 must_p . 该过程为一次 DFS, 遍历所有点与边, 时间复杂度为 $\mathcal{O}(n)$.

接下来计算每个点的 dp_p , 其依赖于所有子节点的 dp_x . 由于每条边最多只会被访问一次, 整个 DP 的计算同样是 $\mathcal{O}(n)$.

因此, 总体时间复杂度为:

$$\mathcal{O}(n + m)$$

该算法空间复杂度也为 $\mathcal{O}(n)$, 用于记录每个点的 dp 和 must 状态.

3.6 伪代码

见附录 1.

4 贪心算法实时策略模拟

4.1 问题描述

任务: 玩家视野受限周围区域, 每次移动时优先选择视野内 ”性价比”(例如单位距离收益最大) 最高的资源, 重复直至无资源可拾取. 输入: 当前玩家位置、视野内的资源信息. 输出: 资源拾取路径.

4.2 贪心策略

1. 优先级驱动的贪心算法

分四个优先级搜索目标, 优先级为:

BOSS / 机关 > 金币 > 通路 > 出口.

每个阶段只考虑当前最高优先级的可行目标.

2. 记忆已探索地图

维持视野为 3×3 范围, 但记录已探索的区域. 利于收集暂时被搁置的资源, 探索待探索的区域. 动态更新环境认知, 优先利用已知信息, 逐步扩展未知区域.

3. 贪心探索路径

优先尝试非陷阱路径, 失败后才降级允许陷阱路径, 当高优先级目标不可达时, 会降级搜索低优先级目标.

4.3 复杂度分析

在最坏情况下, 可能需要探索整个地图遍历所有 n^2 个格子. 每次寻找当前最高优先级的目标, 可以使用一个大根堆, 取得一次最高优先级的元素复杂度为 $\mathcal{O}(\log(n^2)) = \mathcal{O}(\log n)$. 所以总的时间复杂度为 $\mathcal{O}(n^2 \log n)$.

4.4 伪代码

见附录 1.

5 启发式算法实时策略模拟

5.1 原有算法的不足和改进

原本的贪心算法虽然可以找到一个尽可能最大化资源且符合要求的路径, 但是有些情况下, 它会被

陷阱“吓到”而损失了金币, 毕竟谁也不知道一个陷阱的背后是另一块陷阱还是一大片金币. 换言之, 原本的贪心算法十分“保守”.

对此, 我们提出了一种启发式的算法, 该算法在原有贪心的基础上, 引入了估价函数, 对于一片被陷阱挡住的未知区域, 我们可以计算其期望的价值来判断是否“冒险”进入.

5.2 算法描述

为了实现以上改进, 我们优化后的算法有以下几个部分组成:

1. 视野更新机制
2. 优先级策略设计 (策略核心)
3. 预期资源收益估计

5.2.1 视野更新机制

只更新当前位置周围 3×3 的格子, 未探索区域标记为未知. 每当角色移动到新位置, 就拓展一圈视野, 更新地图.

5.2.2 优先级策略设计 (策略核心)

核心目标: 在已知有限信息下, 选择最“有希望”的方向前进.

在没有找全 BOSS 和机关的情况下:

金币 = 通路 = BOSS = 机关 > 陷阱 > 终点.

在找全 BOSS 和机关的情况下:

金币 = 通路 = BOSS = 机关 > 陷阱 > 终点.

对于找全 BOSS 和机关的情况下如果可见且未到达方块中只有陷阱和终点, 我们需要计算陷阱和终点的预期资源收益估计, 如果一个陷阱的预期资源收益估计为正, 我们就会选择去“探索”.

5.2.3 预期资源收益估计

使用类似启发式扩展的方法, 预测从某个点出发能收集多少资源: 可见格子收益为 1 倍; 未知格子按经验折扣估值, 乘一个系数 r .

5.3 复杂度分析

在最坏情况下, 可能需要探索整个地图遍历所有 n^2 个格子. 每次寻找当前最高优先级的目标, 但是由于现在的优先级是实时更新的, 不可以用大根堆, 所以取得一次最高优先级的元素复杂度为 $\mathcal{O}(n^2)$. 所以总的时间复杂度为 $\mathcal{O}(n^4)$.

5.4 实验结果

对于 DP, 贪心, 启发式算法在随机生成的 100 个地图下测试, 计算平均收益.

表1 各算法在不同地图规模下的执行效果比较

算法	7×7	11×11	15×15
贪心策略	85.3	78.2	65.4
动态规划	89.1	83.7	71.8
启发式算法	92.0	85.5	69.2

DP 得分最高: 由于拥有全图信息, 它能保证路径最优;

贪心策略表现最差: 虽快速但容易落入局部最优;

启发式方法综合表现优秀: 仅凭有限视野, 得分接近全图最优解, 展现出“预期收益评估 + 策略优先级”的强大决策能力.

6 回溯法解锁机关密码

6.1 问题描述

输入: 一个 3 位密码锁的位置和线索(如每位密码为素数且不重复、或第 1 位是偶数等)

输出: 密码

6.2 回溯法与剪枝策略

本算法使用回溯法穷举所有三位密码组合, 并通过线索进行剪枝优化, 显著减少无效搜索空间.

剪枝函数判断当前构造出的密码是否满足全部线索条件:

- 若线索为 $[-1, -1]$, 表示密码需为三位不同的素数;
- 若线索长度为 2, 如 $[1, 0]$ 表示第 1 位必须为偶数;
- 若线索长度为 3, 如 $[5, -1, 3]$, 表示第 1 位为 5, 第 3 位为 3.

在回溯函数中, 每填一位数字即调用剪枝函数, 避免无效状态扩展.

6.3 复杂度分析

最坏情况下, 尝试所有 $10^3 = 1000$ 个三位数, 但在有效剪枝下, 实际遍历节点数远小于 1000, 记为 T .

对于每个状态调用一次剪枝函数, 每次最多遍历 $|C|$ 条线索, 故总复杂度为:

$$\mathcal{O}(T \cdot |C|)$$

其中 $|C|$ 为线索数, T 通常远小于 1000.

7 分支限界法优化 BOSS 战斗策略

7.1 问题描述

任务: 在限定回合内击败 BOSS, 寻找最小代价的技能序列.

输入: 玩家剩余资源、BOSS 血量、玩家可用技能(例如, 普通攻击: 伤害5、无冷却; 大招: 伤害10, 冷却2回合).

输出: 最小回合数的技能序列.

7.2 分支限界法与剪枝策略

本策略用于规划击败多个 BOSS 的最优技能施放序列, 采用分支限界 (Branch and Bound) 法进行搜索. 每个状态包括:

- 当前 BOSS 的血量数组;
- 当前正在攻击的 BOSS 编号;
- 当前时间步数 (回合);
- 各技能冷却状态;
- 技能使用历史序列.

通过启发式函数 lowerBound 估计当前状态的最优代价下界 $\text{bound} = \text{turn} + \text{lowerBound}$, 若该值不优于当前最优解则立即剪枝. 估价函数如下:

$$\text{lowerBound} = \left\lceil \frac{\text{剩余所有 BOSS 总血量}}{\text{平均每回合伤害}} \right\rceil$$

同时, 为避免状态重复, 使用哈希函数记录已访问状态 hashState , 并通过集合去重.

7.3 复杂度分析

设 BOSS 数量为 n , 技能数量为 k , 最大每个 BOSS 血量为 H .

最坏情况下, 每一步可以选择 k 个技能或跳过, 最多打掉 nH 点血, 理论状态空间为:

$$\mathcal{O}(k^{nH}).$$

但由于:

- 使用了优先队列, 每次优先扩展估价低的状态;
- 使用了估价剪枝, 大量不优路径被舍弃;
- 使用状态去重, 避免重复搜索;

因此实际状态扩展远少于指数级, 时间复杂度为:

$$\mathcal{O}(T \cdot k)$$

其中 T 为实际扩展的状态数.

8 结论

本文从多个维度探索了经典算法在迷宫探险游戏中的集成应用与落地实现. 通过构建满足特定可达性要求的迷宫生成器, 结合 OpenGL 动态可视化技术, 系统完成了从环境构造到交互展示的闭环.

在策略层面, 设计并实现了针对全局与局部视野的路径规划方案. 全局阶段采用动态规划算法, 有效规避陷阱并最大化资源收益; 局部阶段则结合贪心与启发式策略, 使智能体在不完全信息下作出高收益决策, 实验验证表明本方法在平均得分与稳定性上优于传统贪心策略.

同时, 针对游戏中的逻辑推理与战斗博弈模块, 分别引入回溯法与分支限界法, 成功实现对密码谜题的有效求解与 BOSS 战斗回合数的全局最优控制. 两者均利用剪枝策略显著降低搜索空间, 提升求解效率与实用性.

综上所述, 本文提出的算法融合式游戏设计模式不仅具备良好的通用性与可扩展性, 也为算法教学与智能体训练提供了有价值的范式示例.

致谢 本项目得以顺利完成, 离不开指导老师的耐心指导与宝贵建议. 在整个开发与撰写过程中, 老师在算法设计思路、技术实现细节及报告规范性方面都给予了我们极大的帮助.

同时, 感谢团队成员在项目中通力合作、分工明确, 特别是在迷宫生成、路径规划、策略优化、图形可视化等模块中的共同攻关与调试, 使得项目功能得以完整实现.

此外, 感谢各类开源资源与社区文档为项目提供了良好的参考与支持, 包括 OpenGL 图形库、SHA-256 加密库及 JSON 解析工具等, 为实现各模块功能提供了重要保障.

再次感谢所有支持和帮助本项目顺利完成的老师与同学们!

参考文献

- [1] Three maze generation algorithms: Depth-First Search, Randomized Prim's Algorithm, and Recursive Division. https://blog.csdn.net/qq_41517936/article/details/107047349, 2020-06-30 (三种迷宫算法 (深度优先、随机Prim、递归分割)). https://blog.csdn.net/qq_41517936/article/details/107047349, 2020年6月30日)
- [2] Wang Xiao-Dong. Design and Analysis of Computer Algorithms. 5th ed. Beijing: Publishing House of Electronics Industry, 2018 (in Chinese) (王晓东. 计算机算法设计与分析. 第5版. 北京: 电子工业出版社, 2018)

-
- [3] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. 3rd ed. Cambridge, MA: MIT Press, 2009 <https://zhuanlan.zhihu.com/p/657528677>, 2023-09-21
([树形 DP]树形 DP 的通用思路. <https://zhuanlan.zhihu.com/p/657528677>, 2023年9月21日)
- [4] Aho A. V., Hopcroft J. E., Ullman J. D. The Design and Analysis of Computer Algorithms. Boston, MA: Pearson Education, 2007
- [5] Tree-shaped Dynamic Programming (Tree DP).

附录 1. 伪代码

Algorithm 1 Maze Generation

```

1: function GENMAZE( $n$ )
2:   Initialize maze  $M$  with walls
3:   Set entrance and exit on  $M$  boundary
4:    $P \leftarrow \emptyset$ 
5:    $r \leftarrow \frac{n+1}{2}$ 
6:   GETPATH( $P, 0, 0, r-2, r-2$ )
7:   DESTROYWALL( $P, M$ )
8:   GENERATEELEMENTS( $M$ )
9:   return  $M$ 
10: end function
11: function GETPATH( $P, x_1, y_1, x_2, y_2$ )
12:   if  $x_1 = x_2$  then
13:     for  $i = y_1$  to  $y_2 - 1$  do
14:        $P \leftarrow P \cup \{((x_1, i), (x_1, i+1))\}$ 
15:     end for
16:     return
17:   end if
18:   if  $y_1 = y_2$  then
19:     for  $i = x_1$  to  $x_2 - 1$  do
20:        $P \leftarrow P \cup \{((i, y_1), (i+1, y_1))\}$ 
21:     end for
22:     return
23:   end if
24:   Choose random  $d_i \in [x_1, x_2 - 1]$ ,  $d_j \in [y_1, y_2 - 1]$ 
25:   Open doors around  $(d_i, d_j)$ , add to  $P$ 
26:   GETPATH( $P, x_1, y_1, d_i, d_j$ )
27:   GETPATH( $P, d_i + 1, y_1, x_2, d_j$ )
28:   GETPATH( $P, d_i + 1, d_j + 1, x_2, y_2$ )
29:   GETPATH( $P, x_1, d_j + 1, d_i, y_2$ )
30: end function
31: function DESTROYWALL( $P, M$ )
32:   for all  $e = ((x_1, y_1), (x_2, y_2)) \in P$  do
33:     if  $x_1 = x_2$  then
34:       Remove vertical wall between points in  $M$ 
35:     else
36:       Remove horizontal wall between points in  $M$ 
37:     end if
38:   end for
39: end function
40: function GENERATEELEMENTS( $M$ )
41:   for all empty cell  $c$  do
42:     Place boss, locker, trap, resource with given probabilities
43:   end for
44: end function

```

Algorithm 2 Tree-Based Resource Collection via DFS and DP

```

function TREEDP( $maze$ )
2:    $must[x][y] \leftarrow \mathbf{true}$  if node is on path to end
    $f[x][y] \leftarrow$  maximum resource from subtree rooted at  $(x, y)$ 
4:   function DFS( $x, y, lastx, lasty$ )
   if  $(x, y)$  is end then
6:      $must[x][y] \leftarrow \mathbf{true}$ ; return
   end if
8:    $f[x][y] \leftarrow$  value at  $(x, y)$ 
   for all neighbor  $(u, v)$  of  $(x, y)$  do
10:     if  $(u, v)$  is accessible and  $(u, v) \neq (lastx, lasty)$  then
       DFS( $u, v, x, y$ )
12:     if  $must[u][v]$  then
        $must[x][y] \leftarrow \mathbf{true}$ 
14:        $f[x][y] \leftarrow f[x][y] + f[u][v]$ 
     end if
16:   end if
   end for
18:   for all non-must neighbor  $(u, v)$  do
      $f[x][y] \leftarrow f[x][y] + \max(0, f[u][v])$ 
20:   end for
end function
22: DFS( $start_x, start_y, -1, -1$ )
   GENPATH( $start_x, start_y, -1, -1$ ) // Recover actual path
24: return true
end function

```

Algorithm 3 Path Reconstruction Based on DP Result

```

function GENPATH( $x, y, lastx, lasty$ )
    Append ( $x, y$ ) to path
3:   if  $must[x][y]$  then
        for all non-must neighbor ( $u, v$ ) with
         $f[u][v] > 0$  do
            GENPATH( $u, v, x, y$ ); Append ( $x, y$ ) to
            path
6:   end for
        for all must neighbor ( $u, v$ ) do
            GENPATH( $u, v, x, y$ )
9:   end for
    else
        for all non-must neighbor ( $u, v$ ) with
         $f[u][v] > 0$  do
12:      GENPATH( $u, v, x, y$ ); Append ( $x, y$ ) to
        path
        end for
    end if
15: end function

```

Algorithm 4 SmartRunner run() Main Execution Flow

```

function SMARTRUNNER.RUN
    Initialize  $currentMaze$  with unknowns,
    mark  $isPushed$  to false
    Locate start position ( $x, y$ ) where
     $originMaze[x][y] = S$ 
4:   BUILDTREE( $x, y$ ) to build DFS tree and depths
    Set  $currentMaze[i][j] = originMaze[i][j]$ 
    for  $i, j \in 3 \times 3$  neighborhood of ( $x, y$ )
    Mark start as pushed, add ( $x, y$ ) to queue  $q$ ,
    path, and initial point
    while  $q$  is not empty do
8:       Select ( $tx, ty$ )  $\in q$  with highest priority via
        LESS( $tx, ty, u, v$ )
        Pop ( $tx, ty$ ) from  $q$ , set  $x \leftarrow tx, y \leftarrow ty$ 
        Add ( $x, y$ ) to point, add value to  $w$ 
        if ( $x, y$ ) is  $E$  and  $B, L$  are both found then
12:            break
        end if
        Mark  $B, L$  if met; mark surroundings as
        known in  $currentMaze$ 
        for all adjacent ( $tx, ty$ ) of ( $x, y$ ) not yet
        pushed do
16:            Mark as pushed and add to  $q$ 
        end for
    end while
    for  $i = 1$  to  $|point| - 1$  do
20:      GENPATH(point[i-1], point[i]) and merge
        into final path
    end for
    return true
end function

```



张佳栋 负责核心算法模块的实现, 包括树形动态规划路径规划、启发式算法等. 同时编写了可视化部分代码. 以及负责汇报和报告的编写.

工作量占比 40%.



方维 负责迷宫数据分支构造与可视化导出, 以及最优路径和得分的保存. 同时协助调试和测试完整流程.

工作量占比 20%.



逯熙钰 负责有限视野下的局部路径策略, 设计并实现了基于预期收益的贪心路径选择算法, 提升了实时策略决策的效率与得分表现.

工作量占比 20%.



姜舜航 负责三位密码的回溯求解以及 BOSS 战分支限界法等关键功能. 同时协助调试和测试完整流程.

工作量占比 20%.